# MidwayJS开发框架

## 前言

上一次分享了《Node.js的热门框架：Nest.js特性、常用功能和代码示例介绍》

本次将分享下 Node.js 的另一个热门开发框架 MidwayJS。

你可能没有听过 `MidwayJS` ，但你一定听过或者使用过 `Koa` / `Express` / `Egg` ， `Egg` 是基于 `Koa` 并在其能力上做了增强，同时它又能作为一款定制能力强的基础框架，来使得你能基于自己的技术架构封装出一套适合自己业务场景的框架。 `MidwayJS` 正是基于 `Egg` ，但在 `Egg` 的基础上做了一些较大的变动：

1. 更好的TS支持，可以说写 MidwayJS 比较舒服的一个地方就是它的 TypeScript 支持了，与 TypeORM 这种TS支持好的框架协作起来更是愉悦。
2. IoC机制的路由，MidwayJS 同时保留了 Egg 的路由能力，即 `src/app/router.ts` 的路由配置方式
3. 生态复用，Egg与Koa的中间件大部分能在 MidwayJS 应用中完美兼容，少部分暂不支持的也由官方团队在快速兼容中。
4. 稳定支持，MidwayJS 至今仍在快速发展迭代，同时也在阿里内部作为 Serverless 基建的重要成员而受到相当的重视，所以你不用担心它后续的维护情况。

---

MIDWAY

For Serverless And Application

# 一、MidwayJS 简介

1. **什么是 MidwayJS?**
   - MidwayJS 基于 TypeScript 的 Node.js 企业级框架，结合了 `面向对象（OOP + Class + IoC）` 与 `函数式（FP + Function + Hooks）` 两种编程范式，并在此之上支持了 Web / 全栈 / 微服务 / RPC / Socket / Serverless 等多种场景，致力于为用户提供简单、易用、可靠的 Node.js 服务端研发体验。
   - 兼容 Express/Koa 生态，适合传统应用和云原生场景。
2. **核心特性**
   - **依赖注入（DI）**：解耦代码，便于测试和维护。
   - **模块化**：通过 `@Provide()` 和 `@Inject()` 管理组件。
   - **多运行时支持**：MidwayJS 能够在不同的运行时环境（如 HTTP 服务、WebSocket、Serverless 无服务架构等）中以统一的开发模式和代码结构运行，开发者无需为不同场景重写核心逻辑。这一特性是 MidwayJS 作为“全栈框架”的核心优势之一。
3. **对比 NestJS**
   - 更轻量，中文文档友好，适合国内团队。
   - 默认集成 **Egg.js** 的插件生态。
   - 类似的多运行时支持，但 MidwayJS 更贴近国内云服务生态（如阿里云）。

---

# 二、MidwayJS 的特点与优势

## # 特点

1. **基于TypeScript：**Midway 采用了TypeScript作为开发语言，可以提供更好的类型检查和代码提示，从而减少开发者的错误和调试时间。
2. **面向对象的设计：**Midway 采用了面向对象的设计思想，将应用程序分解成多个模块，每个模块都具有独立的职责和功能，可以更好地管理和维护代码。
3. **高效的依赖注入：**Midway 提供了高效的依赖注入机制，可以自动管理模块之间的依赖关系，减少了手动管理依赖关系的工作量。
4. **插件化的架构：**Midway 采用了插件化的架构，可以方便地添加和移除插件，从而扩展应用程序的功能和特性。
5. **集成化的开发体验：**Midway 提供了集成化的开发体验，包括代码生成、调试、测试、部署等多个方面，可以大大提高开发效率和质量。

## # 优势

1. 稳定支持，Midway 框架是在阿里内部已经使用 5 年以上的 Node.js 框架，同时也在阿里内部作为 Serverless 基建的重要成员而受到相当的重视，所以你不用担心它后续的维护情况，有着持续维护的团队做后盾。
2. 丰富的组件和扩展能力，例如数据库，缓存，定时任务，进程模型，部署以及 Web Socket 甚至 Serverless 等新场景的支持。
3. 一体化调用方案可以方便快捷和前端页面协同开发。
4. 统一开发体验，使用一致的依赖注入、配置管理、装饰器语法，降低学习成本。

---

# 三、MidwayJS 的多编程范式

## # 面向对象（OOP + Class + IoC）

基于面向对象，开发路由的示例

面向对象（OOP + Class + IoC）

```
// src/controller/home.ts
import { Controller, Get } from '@midwayjs/core';
import { Context } from '@midwayjs/koa';
```

```

@Controller('/')
export class HomeController {

  @Inject()
  ctx: Context

  @Get('/')
  async home() {
    return {
      message: 'Hello Midwayjs!',
      query: this.ctx.ip
    }
  }
}
```

**面向对象编程的区别与体现：**

- 在 MidwayJS 中，OOP 是其核心思想之一。它通过强大的 **依赖注入 (Dependency Injection - DI)** 容器和 **装饰器 (Decorators)** 机制来实现。
- 开发者使用 Class（类）来定义控制器 (Controller)、服务 (Service)、中间件 (Middleware)、配置 (Config) 等各种组件。
- 利用 `@Controller`，`@Service`，`@Provide`，`@Inject` 等装饰器来标识组件、注入依赖。
- 强调封装、继承（虽然在 Node.js 业务开发中继承不如组合常用，但类本身支持）、多态。
- 代码组织围绕“对象”或“组件”展开，每个对象负责特定的职责，并通过接口或类型定义规范交互。

**面向对象编程的优势：**

- **结构清晰，模块化高：**代码被组织成独立的类，职责明确，易于理解和管理大型项目。
- **可维护性好：**通过封装，修改一个类的内部实现不容易影响到外部使用者。
- **可测试性强：**依赖注入使得单元测试时可以方便地 Mock (模拟) 依赖，降低组件间的耦合。
- **复用性：**通过定义类，可以在不同地方创建该类的实例并复用其逻辑。
- **类型安全：**结合 TypeScript，OOP 提供了强大的类型检查，减少运行时错误。
- **符合传统后端开发习惯：**对于有 Java/Spring, C#/.NET 等背景的开发者而言，OOP 风格更易于上手。

# 函数式（FP + Function + Hooks）

基于函数式，开发路由接口的示例

函数式（FP + Function + Hooks）

```
// src/api/index.ts
import { useContext } from '@midwayjs/hooks';
import { Context } from '@midwayjs/koa';

export default async function home () {
  const ctx = useContext<Context>()

  return {
    message: 'Hello Midwayjs!',
    query: ctx.ip
  }
}
```

**函数式编程的区别与体现：**

- MidwayJS **并没有强制**使用纯函数式编程，但它 **鼓励和方便**开发者在特定场景下采用函数式风格。
- 体现在编写**无副作用的纯函数**来处理数据转换、逻辑计算。
- Middleware (中间件) 可以很自然地使用函数式风格编写，形成函数链。
- 在 Service 内部的具体方法实现中，可以使用函数组合、高阶函数等 FP 概念。
- 强调数据不可变性（虽然 JavaScript/TypeScript 本身是可变的，但开发者可以遵循这个原则）。

**函数式编程的优势：**

- **代码简洁，易于理解：**纯函数只依赖输入并产生输出，没有外部依赖和副作用，逻辑清晰明了。
- **可测试性极强：**纯函数给定相同的输入总会得到相同的输出，测试中只需关注输入和输出。
- **易于并行/并发处理：**由于无副作用和状态，函数更容易在并发环境中安全执行。
- **可组合性强：**函数可以像乐高积木一样自由组合，构建复杂的逻辑。
- **减少 Bug：**避免了共享状态和副作用带来的意外修改，减少了潜在的 Bug。
- **适用于特定场景：**例如数据处理、转换、验证等场景，函数式风格往往更具表现力。

## # MidwayJS 多编程范式的优势

1. **各取所长：**开发者可以根据任务的性质选择最合适的范式。复杂的、需要结构化和DI管理的业务逻辑适合 OOP；纯粹的数据转换、无状态的计算或可复用的管道处理适合 FP；而框架层面的配置和组件定义则大量利用声明式。

2. **降低耦合与提高可测试性：**OOP 的 DI 和 FP 的纯函数特性都极大地提升了代码的解耦程度和单元测试的便利性。
3. **提高开发效率与代码质量：**结合 TypeScript 和装饰器，提供了强大的类型检查和元编程能力，减少错误，提高开发效率。
4. **适应不同背景的开发者：**无论是熟悉 OOP 的传统后端开发者，还是喜欢 FP 风格的开发者，都能在 MidwayJS 中找到舒适的开发方式。
5. **构建更健壮、可维护的应用：**通过合理地结合使用这些范式，可以构建出结构清晰、易于理解、易于测试和维护的复杂应用系统。

总而言之，MidwayJS 的多编程范式支持不是简单地堆砌概念，而是在其核心的 OOP + DI 架构基础上，融入并鼓励使用 FP、声明式等其他范式的优点，旨在帮助开发者构建更高效、更稳定、更易于维护的企业级 Node.js 应用。

---

# 四、IoC、DI、容器

## # IoC的全称为 Inversion of Control，意为控制反转

它是OOP中的一种原则（虽然不在n大设计模式中，但实际上IoC也属于一种设计模式），它可以很好的解耦代码。

在不使用IoC的情况下，我们很容易写出来这样的代码**（传统方式）**

IoC

```
import { A } from './modA';
import { B } from './modB';

class C {
  constructor() {
      this.a = new A();
      this.b = new B();
    }
}
```

当一个对象需要依赖另一个对象时，它通常会自己去创建或查找这个依赖对象（比如使用 `new` 关键字创建一个实例，或者通过工厂方法获取）。控制权在使用者（调用者）手中。

乍一看可能没什么，但实际上类C会强依赖于类A、类B，造成模块之间的耦合。要解决这个问题，我们可以这么做：用一个第三方容器来负责管理容器，当我们需要某个实例时，由这个容器来替我们实例化并交给我们实例。以 `Injcetion` 为例（**IoC 方式**）

IoC

```
import { Container } from 'injection';
import { A } from './A';
import { B } from './B';

const container = new Container();
container.bind(A);
container.bind(B);

class C {
  constructor() {
    this.a = container.get(A);
    this.b = container.get(B);
  }
}
```

对象不再负责创建或查找它的依赖。而是由一个外部实体（通常是框架或容器）来负责创建依赖对象，并将这些依赖“给予”需要它们的对象。控制权从使用者手中转移到了外部实体（容器）。

## # DI的全称为Dependency Injection，即依赖注入

依赖注入是控制反转最常见的一种应用方式，就如它的名字一样，它的思路就是在对象创建时自动注入依赖对象。再以 `Injection` 的使用为例：

DI

```
// provide 意为当前对象需要被绑定到容器中
// inject 意为去容器中取出对应的实例注入到当前属性中
@provide()
export class UserService {
  @inject()
  userModel: UserModel;

  async getUser(userId: string) {
    return await this.userModel.get(userId);
  }
}
```

# 容器 (IoC Container / DI Container)

它是负责**管理**对象（组件）的生命周期、创建对象、处理对象之间的**依赖关系**，并执行**依赖注入**的“幕后工作者”。

你可以把容器想象成一个智能工厂或服务注册中心。你告诉它有哪些组件（类），这些组件依赖于什么，它就会帮你创建这些组件的实例，并在需要时把它们注入到其他组件中。

# IoC、DI、容器三者关系总结

- **IoC 是目标/原则：**控制权的转移。
- **DI 是手段/模式：**通过注入的方式来实现控制反转。
- **容器 是工具/实现：**一个软件组件，它负责执行 DI，从而实现了 IoC 原则，并管理对象的生命周期和依赖关系。

# 面向对象的IoC与函数式编程的Hooks比较

## 相似点

1. **控制权转移：**
   - 两者都实现了控制权的反转
   - IoC将对象创建和依赖管理的控制权从类内部转移到容器
   - Hooks将状态管理和副作用控制从组件内部转移到React运行时
2. **解耦：**都实现了关注点分离
   - IoC解耦了对象和使用它的客户端
   - Hooks解耦了状态逻辑和UI渲染

## 主要区别

| 方面 | 面向对象IoC | 函数式Hooks |
| --- | --- | --- |
| 范式基础 | 基于类和对象 | 基于函数和闭包 |
| 实现方式 | 依赖注入/服务定位 | 函数组合和闭包 |
| 典型用例 | 企业应用架构 | React组件状态管理 |
| 运行时特性 | 通常静态(编译时)决定 | 更动态(运行时组合) |

# 五、代码示例介绍

使用 `pnpm create midway` 来创建项目

项目结构介绍

```
├─src                      # 源码目录
│  ├─config                # 配置
│  ├─controller            # 控制器
│  ├─entity                # 数据对象模型
│  ├─filter                # 过滤器
│  ├─middleware            # 中间件
│  ├─service               # 服务类
│  ├─configurations.ts     # 服务生命周期管理及配置
│  └─interface.ts          # 接口定义
├─test                     # 测试类目录
├─bootstrap.js             # 启动入口
├─package.json             # 包管理配置
├─tsconfig.json            # TypeScript 编译配置文件
```

## # 编写 Controller

如果你熟悉 Web 开发或 MVC，就知道第一步我们需要编写 Controller 和 Router。

在脚手架创建的文件中，我们已经有了一些文件，我们暂时忽略他们。

在 `controller` 目录中，新建一个 `src/controller/weather.controller.ts` 文件，内容如下。

Controller => Router

```
// src/controller/weather.controller.ts
import { Controller, Get } from '@midwayjs/core';

@Controller('/')
export class WeatherController {
  // 这里是装饰器，定义一个路由
  @Get('/weather')
  async getWeatherInfo(): Promise<string> {
    // 这里是 http 的返回，可以直接返回字符串，数字，JSON，Buffer 等
```

```
    return 'Hello Weather!!!';
  }
}
```

现在我们可以通过访问 `/weather` 接口返回数据了。

`@controller` 装饰器标注了控制器，装饰器有一个可选参数，用于进行路由前缀，这样控制器下面的所有路由都会带上这个前缀。

## # 添加参数处理

在示例中，我们需要一个 URL 参数来动态展示不同城市的天气。

通过添加 `@Query` 装饰器，我们可以获取到 URL 上的参数。

Controller => @Query

```
// src/controller/weather.controller.ts
import { Controller, Get, Query } from '@midwayjs/core';

@Controller('/')
export class WeatherController {
  @Get('/weather')
  async getWeatherInfo(@Query('cityId') cityId: string): Promise<string> {
    return cityId;
  }
}
```

除了 `@Query` 装饰器，Midway 也提供了其他请求参数的获取（@body，@heander，@Param ...），具体可以查看官方的 路由和控制 文档。

## # 编写 Service

在实际项目中，Controller 一般用来接收请求参数，校验参数，不会包括特别复杂的逻辑，复杂而复用的逻辑，我们应该封装为 Service 文件。

我们来添加一个 Service 用来获取天气信息，其中包括一个 http 请求，获取远端的数据。

Service

```
// src/service/weather.service.ts
import { Provide, makeHttpRequest } from '@midwayjs/core';

export interface WeatherInfo {
```

```
  weatherinfo: {
    city: string;
    cityid: string;
    temp: string;
    WD: string;
    WS: string;
    SD: string;
    AP: string;
    njd: string;
    WSE: string;
    time: string;
    sm: string;
    isRadar: string;
    Radar: string;
  };
}

@Provide()
export class WeatherService {
  async getWeather(cityId: string): Promise<WeatherInfo> {
    const result = await makeHttpRequest<WeatherInfo>(
      `https://midwayjs.org/resource/${cityId}.json`,
      {
        dataType: 'json',
      }
    );

    if (result.status === 200) {
      return result.data as WeatherInfo;
    }
  }
}

```

💡 1. 这里使用 `@Provide` 装饰器修饰类，便于后续 Controller 注入该类
2. makeHttpRequest 方法是 Midway 内置的 http 请求方法，更多参数请查看 文档

最后，修改下 Controller 文件

Controller => Service

```
import { Controller, Get, Inject, Query } from '@midwayjs/core';
import { WeatherInfo } from '../service/weather.service';
```

```
import { WeatherService } from '../service/weather.service';

@Controller('/')
export class WeatherController {
  @Inject()
  weatherService: WeatherService;

  @Get('/weather')
  async getWeatherInfo(@Query('cityId') cityId: string): Promise<WeatherInfo> {
    return this.weatherService.getWeather(cityId);
  }
}
```

> 💡 这里使用 `@Inject` 装饰器注入 `Service`，是 Midway 依赖注入的标准用法，可以查看这里了解更多

使用服务的过程分为几部分：

1. 使用 `@Provide` 装饰器暴露你的服务
2. 在调用的代码处，使用 `@Inject` 装饰器注入你的服务
3. 调用注入服务，执行对应的方法

Midway 的核心 “依赖注入” 容器会**自动关联**你的控制器（Controller） 和服务（Service），在运行过程中**会自动初始化**所有的代码，你**无需手动初始化**这些 Class。

# `@Provide` 与 `@Inject`

1. `@Provide` 装饰器
   - 标记一个类，表明它是一个可以被 MidwayJS 的 IoC 容器所管理和提供的组件。一旦一个类被 `@Provide` 标记，容器就会负责它的实例化、生命周期管理，并使其可以被 `@Inject` 注入。
2. `@Inject` 装饰器
   - 在一个类中，标记某个属性或构造函数参数需要从 IoC 容器中获取一个依赖项的实例，然后由容器负责将对应的实例“注入”到这个位置。

> ❗ `@Provide` 和 `@Inject` 装饰器是成对出现的，@Inject 的类中，必须有对应的 `@Provide` 才会生效。

看到这里，你可能会有一些疑惑，为什么服务（Service）上有一个 `@Provide` 装饰器，但是控制器（Controller) 上没有。

事实上，控制器（Controller） 上也有这个装饰器，只是在新版本中，Controller 包含了 Provide 的功能。如果你不确定什么时候可以隐藏，可以都写上。

你如果不写，默认等价于下面的代码。

```
@Previde
@Provide()
@Controller('/')
export class WeatherController {
```

### `@Provide` 和 `@Inject` 如何协同工作?

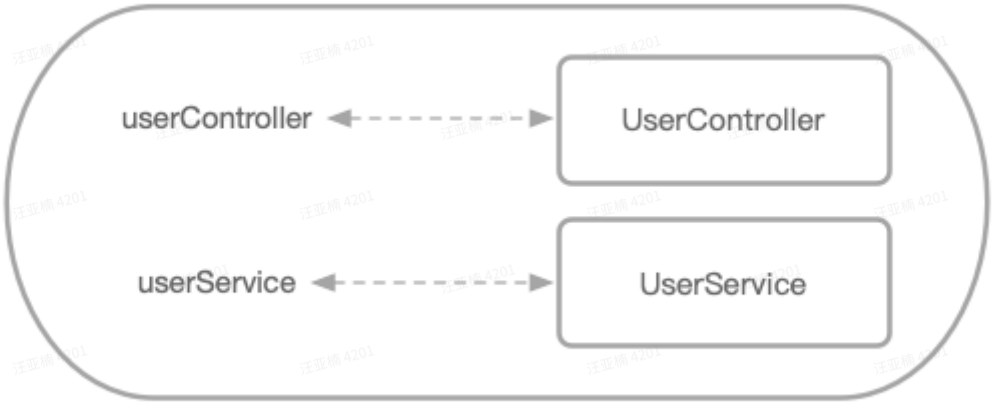
1. **注册阶段：** MidwayJS 启动时，IoC 容器扫描你的代码，发现所有用 `@Provide` 标记的类。它将这些类及其可能的别名注册到其内部的一个“组件注册表”中。
2. **解析/注入阶段：** 当容器需要实例化一个类（比如一个 `@Controller` ）时，它会检查这个类内部是否有属性或构造函数参数使用了 `@Inject` 。
3. 对于每一个 `@Inject` ，容器会根据指定的类型（如果 `@Inject()` 无参）或指定的别名（如果 `@Inject('别名')` 有参），去“组件注册表”中查找对应的类定义。
4. 容器会负责实例化找到的类（如果还没有实例，并且根据其生命周期规则需要创建新实例），并递归处理这个依赖项自身的 `@Inject` 。
5. 最后，容器将解析并创建好的依赖项实例，“注入”到原始对象中被 `@Inject` 标记的位置（属性或构造函数参数）。

## # 依赖注入原理

我们以下面的伪代码举例，在 Midway 体系启动阶段，会创建一个依赖注入容器（MidwayContainer），扫描所有用户代码（src）中的文件，将拥有 `@Provide` 装饰器的 Class，保存到容器中。

```
Midway
/***** 下面为 Midway 内部代码 *****/
const container = new MidwayContainer();
container.bind(UserController);
container.bind(UserService);
```

这里的依赖注入容器类似于一个 Map。Map 的 key 是类对应的标识符（比如 **类名的驼峰形式字符串**），Value 则是 **类本身**。




在请求时，会动态实例化这些 Class，并且处理属性的赋值，比如下面的伪代码，很容易理解。

代码块

```
/***** 下面为依赖注入容器伪代码 *****/
const userService = new UserService();
const userController = new UserController();

userController.userService = userService;
```

这样处理后，我们就能拿到完整的 `userController` 对象了，实际的代码会稍微不一样（见下方）。

代码块

```
// MidwayContainer 有 getAsync 方法，用来异步处理对象的初始化（很多依赖都是有异步初始化的需求），自动属性赋值，缓存，返回对象，将上面的流程合为同一个。


/***** 下面为依赖注入容器内部代码 *****/

// 自动 new UserService();
// 自动 new UserController();
// 自动赋值 userController.userService = await container.getAsync(UserService);

const userController = await container.getAsync(UserController);
await userController.handler();  // output 'world'
```

以上就是依赖注入的核心过程

> 💡 此外，这里还有一篇名为 《这一次，教你从零开始写一个 IoC 容器》的文章，欢迎扩展阅读。

---

## 六、模板渲染

从这里开始，我们需要用到一些 Midway 的扩展能力。

Midway 对应的扩展包我们称为 “组件”，也是标准的 npm 包。

这里我们需要用到 `@midwayjs/view-nunjucks` 视图引擎集成组件。

这个库封装了模板引擎，使其能够无缝地在 Midway 应用中使用。

view-nunjucks

```
pnpm i @midwayjs/view-nunjucks --save
```

安装完成后，我们在 `src/configuration.ts` 文件中启用组件。

Configuration

```
// ...
import * as view from '@midwayjs/view-nunjucks';

@Configuration({
  imports: [
    koa,
    // ...
    view
  ],
  importConfigs: [join(__dirname, './config')],
})
export class MainConfiguration {
  // ...
}
```

> 💡 1. `configuration` 文件是 Midway 的生命周期入口文件，承担了组件开关，配置加载和生命周期管理的作用

2. `imports` 就是用来导入（开启）组件的方法

在 `src/config/config.default.ts` 中配置组件，指定为 `nunjucks` 模板。

Config

```
import { MidwayConfig } from '@midwayjs/core';

export default {
  // ...
  view: {
    defaultViewEngine: 'nunjucks',
  },
} as MidwayConfig;
```

在根目录（非 src 里）添加模板 `view/info.html` 文件，内容如下：

代码块

```
<!DOCTYPE html>
<html>
  <head>
    <title>天气预报</title>
    <style>
      .weather_bg {
        background-color: #0d68bc;
        height: 150px;
        color: #fff;
        font-size: 12px;
        line-height: 1em;
        text-align: center;
        padding: 10px;
      }

      .weather_bg label {
        line-height: 1.5em;
        text-align: center;
        text-shadow: 1px 1px 1px #555;
        background: #afdb00;
        width: 100px;
        display: inline-block;
        margin-left: 10px;
      }

```

```
26        .weather_bg .temp {
27          font-size: 32px;
28          margin-top: 5px;
29          padding-left: 14px;
30        }
31        .weather_bg sup {
32          font-size: 0.5em;
33        }
34      </style>
35    </head>
36    <body>
37      <div class="weather_bg">
38        <div>
39          <p>
40            {{city}} ({{WD}}{{WS}})
41          </p>
42          <p class="temp">{{temp}}<sup>℃</sup></p>
43          <p>
44            气压<label>{{AP}}</label>
45          </p>
46          <p>
47            湿度<label>{{SD}}</label>
48          </p>
49        </div>
50      </div>
51    </body>
52  </html>
53
```

同时，我们调整 Controller 的代码，将返回 JSON 变为模板渲染。

weather.controller.ts

```
1  import { Controller, Get, Inject, Query } from '@midwayjs/core';
2  import { WeatherService } from '../service/weather.service';
3  import { Context } from '@midwayjs/koa';
4
5  @Controller('/')
6  export class WeatherController {
7    @Inject()
8    weatherService: WeatherService;
9
10   @Inject()
11   ctx: Context;
12
13   @Get('/weather')
```

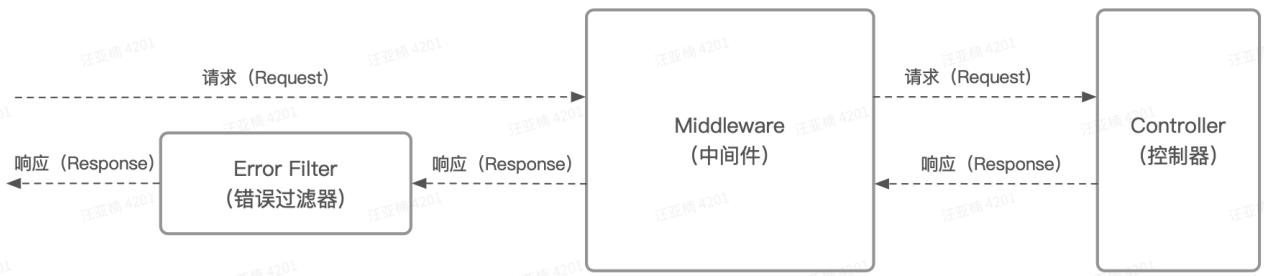
```
  async getWeatherInfo(@Query('cityId') cityId: string): Promise<void> {
    const result = await this.weatherService.getWeather(cityId);
    if (result) {
      await this.ctx.render('info', result.weatherinfo);
    }
  }
}
```

到这一步，我们访问 `http://127.0.0.1:7001/weather?cityId=101010100` 已经可以看到渲染的模板内容了。

---

# 七、错误处理

最后，我们还有一些异常的逻辑需要处理，Midway 提供了一个内置的异常处理器，负责处理应用程序中所有未处理的异常。当您的应用程序代码抛出一个异常处理时，该处理器就会捕获该异常，然后等待用户处理。




为此，我们需要定义一个我们自己的业务错误，创建一个 `src/error/weather.error.ts` 文件。

weather.error.ts

```
import { MidwayError } from '@midwayjs/core';

export class WeatherEmptyDataError extends MidwayError {
  constructor(err?: Error) {
    super('weather data is empty', {
      cause: err,
```

```
    });
    if (err?.stack) {
      this.stack = err.stack;
    }
  }
}
```

然后，我们调整 Service 代码抛出异常。

src/service/weather.service.ts

```
import { Provide, makeHttpRequest } from '@midwayjs/core';
import { WeatherInfo } from '../interface';
import { WeatherEmptyDataError } from '../error/weather.error';

@Provide()
export class WeatherService {
  async getWeather(cityId: string): Promise<WeatherInfo> {
    if (!cityId) {
      throw new WeatherEmptyDataError();
    }

    try {
      const result = await makeHttpRequest<WeatherInfo>
(`https://midwayjs.org/resource/${cityId}.json`, {
        dataType: 'json',
      });
      if (result.status === 200) {
        return result.data as WeatherInfo;
      }
    } catch (error) {
      throw new WeatherEmptyDataError(error);
    }
  }
}
```

💡

1. 将 http 的调用请求进行错误捕获，将错误包裹，返回一个我们系统的业务错误
2. 如有必要，我们可以定义更多的错误，分配错误 Code 等

到这一步，我们还需要将异常进行业务处理，比如有多个位置抛出 `WeatherEmptyDataError` 时，我们需要统一的格式返回。

错误处理器可以完成这个功能，我们需要创建一个 `src/filter/weather.filter.ts` 文件，内容如下：

src/filter/weather.filter.ts

```
import { Catch } from '@midwayjs/core';
import { Context } from '@midwayjs/koa';
import { WeatherEmptyDataError } from '../error/weather.error';

@Catch(WeatherEmptyDataError)
export class WeatherErrorFilter {
  async catch(err: WeatherEmptyDataError, ctx: Context) {
    ctx.logger.error(err);
    return '<html><body><h1>weather data is empty</h1></body></html>';
  }
}
```

然后应用到当前的框架中。

代码块

```
import { Configuration, App } from '@midwayjs/core';
import * as koa from '@midwayjs/koa';
import { WeatherErrorFilter } from './filter/weather.filter';
// ...

@Configuration({
  // ...
})
export class MainConfiguration {
  @App()
  app: koa.Application;

  async onReady() {
    // ...

    // add filter
    this.app.useFilter([WeatherErrorFilter]);
  }
}
```

异常处理的更多信息，可以查阅 文档。

至此，你已对 Midway 有了初步了解。最后我们来简单的回顾一下。

1. 使用 `pnpm create midway` 来创建项目
2. 使用 `@Controller` 装饰器定义路由和控制器类
3. 使用 `@Query` 获取请求参数
4. 使用 `@Provide` 和 `@Inject` 注入服务类
5. 使用 `imports` 启用组件，并配置了模板引擎
6. 自定义了错误，并使用错误过滤器来拦截错误，返回自定义的数据

---

END